

How to Design a Curriculum

Matthias Felleisen, PLT

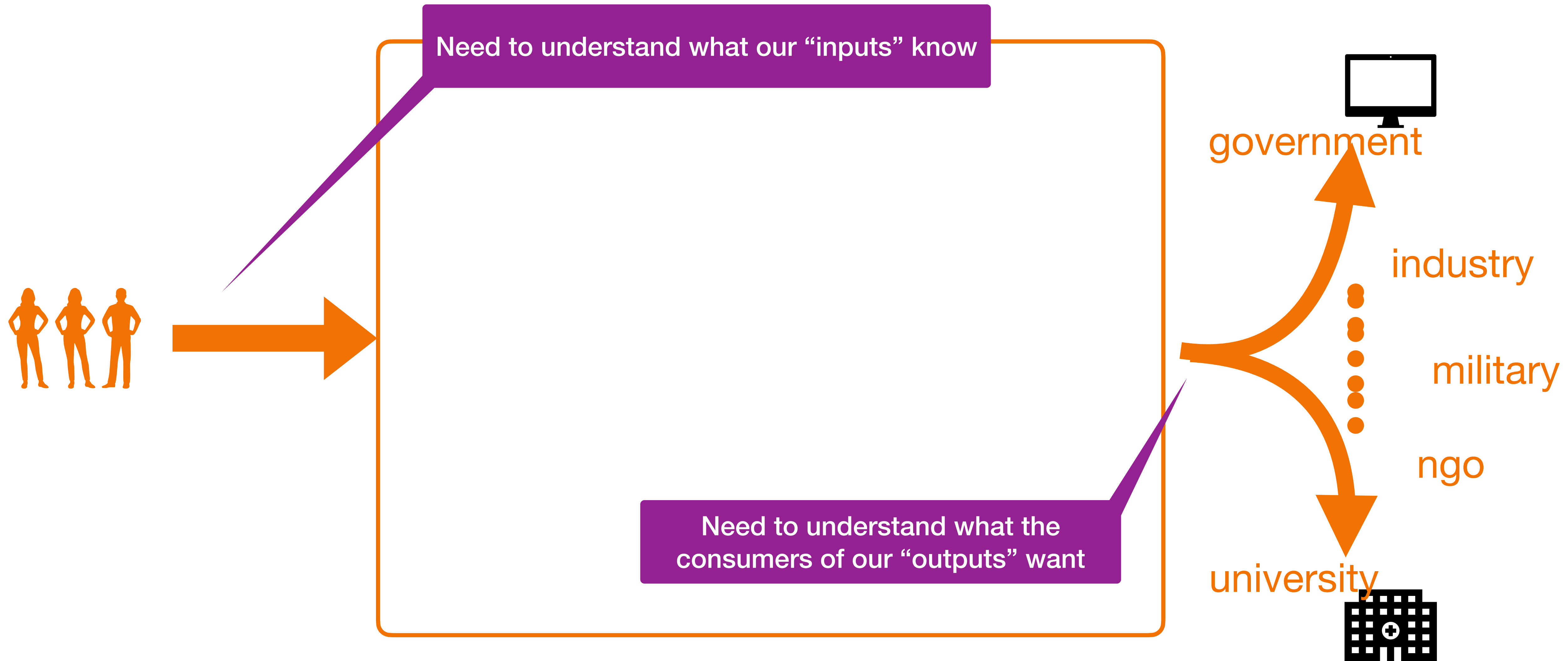
How I Came to This Conclusion: Failures, Successes

- 1992 — first ideas for a “Fundamentals 1” course; *total failure with high enthusiasm*
- 1994 — a radical alternative: “Fundamentals 1” step-wise define with checkable products
- 1995 — *high school teachers and students can solve word problems with the design recipe*
- 1997 — first ideas for a “Software Development” course; insight: principles don’t “stick” with one course
- 2001 — *ouch — the vast majority of seniors @ Northeastern cannot develop software for 30K p.a.*
- 2004 — Fundamentals 1 and 2 plus Software Development; repeat uses of “systematic design” in alternative PLs, at scale
- 2005 — Logic, because students need to learn how to express specifications as code
- 2007 — the graduate dean begs for a transfer of the new core to the MS level
- 2015 — Object-Oriented Design becomes the essential bridge to scale up “systematic design” (“Sw Dev” in one PL)
 - Thanks to Jesse Tov and even more to Ben Lerner for taking my design sketches and making them real
- 2016 — scaling F 1 to 650 students per fall semester

The College Curriculum



The College Curriculum



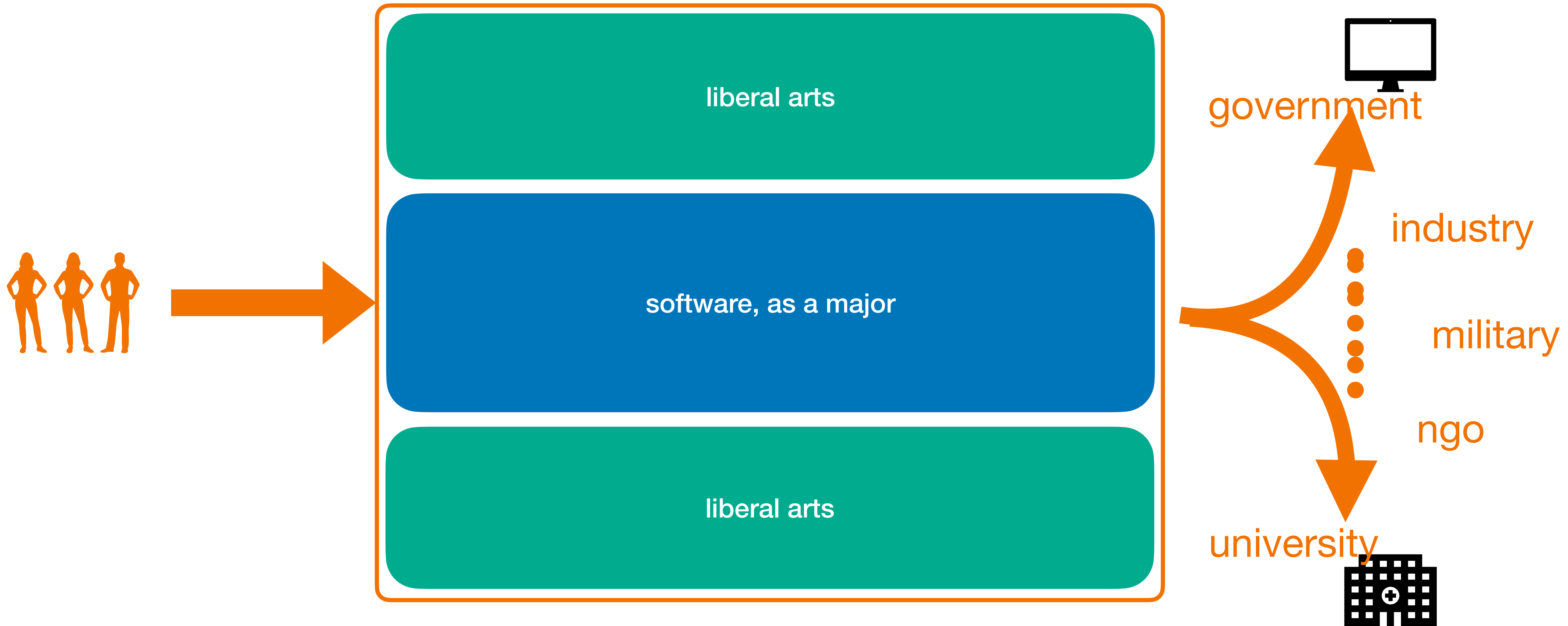
Basic Assumptions About Students & Colleges

(1) Students wish to receive an **education** and get **prepared for work**.

(2) Colleges wish to **educate for life** and providing **work-oriented education**.

So let's focus on “consumers” who offer jobs that *truly* need a college degree, including education and training.

The College Curriculum



The Designer's Choice

Date: February 8, 9:59AM

Good morning Matthias

...

I studied at the University of ..., where the computer science curriculum was structured more like a course catalog with credits rather than a well-organized sequence with a clear learning path. Looking back, I regret not having a more structured foundation, as I still feel like I missed some key fundamentals.

...

Best regards,

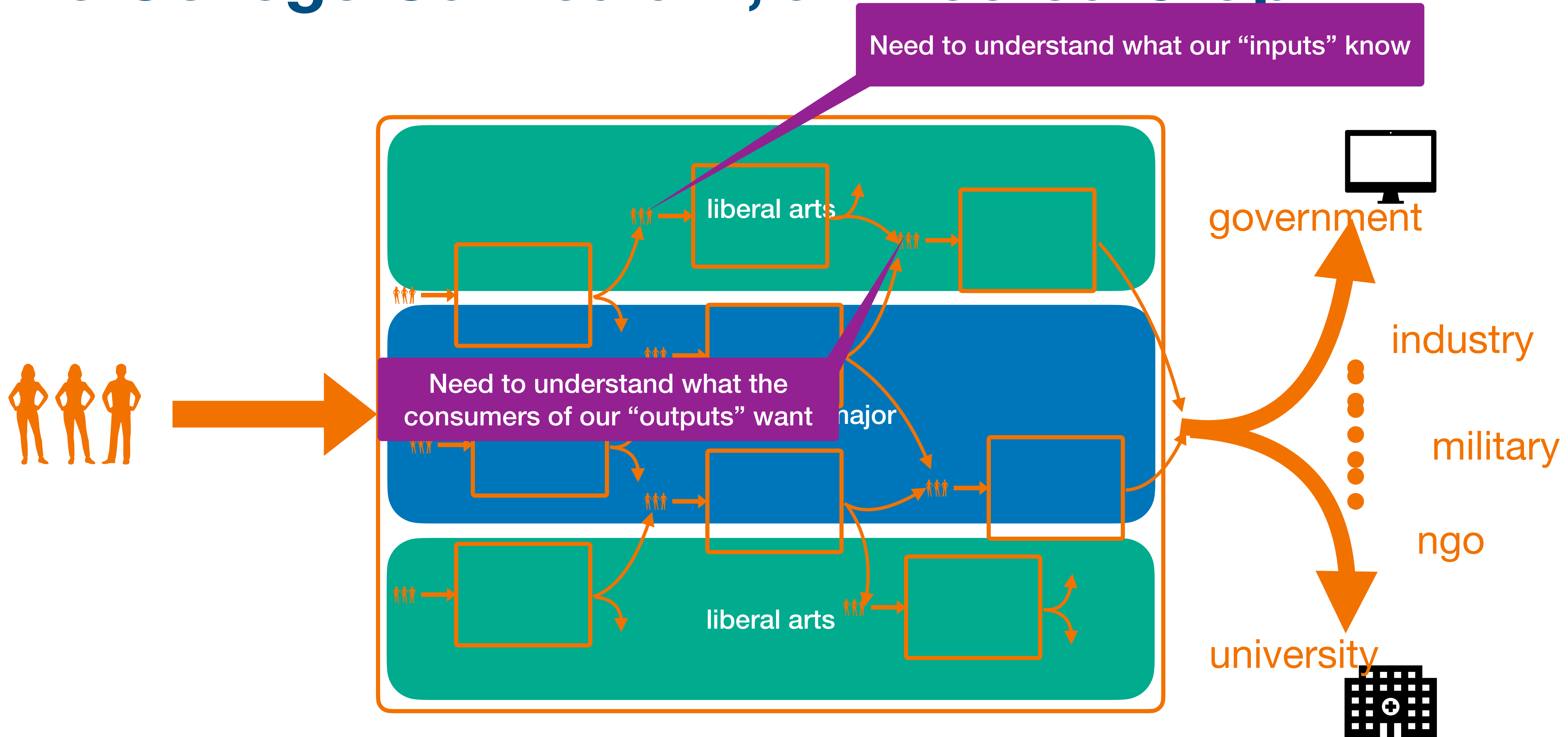
Florian B.

Program and Transformation Coordinator

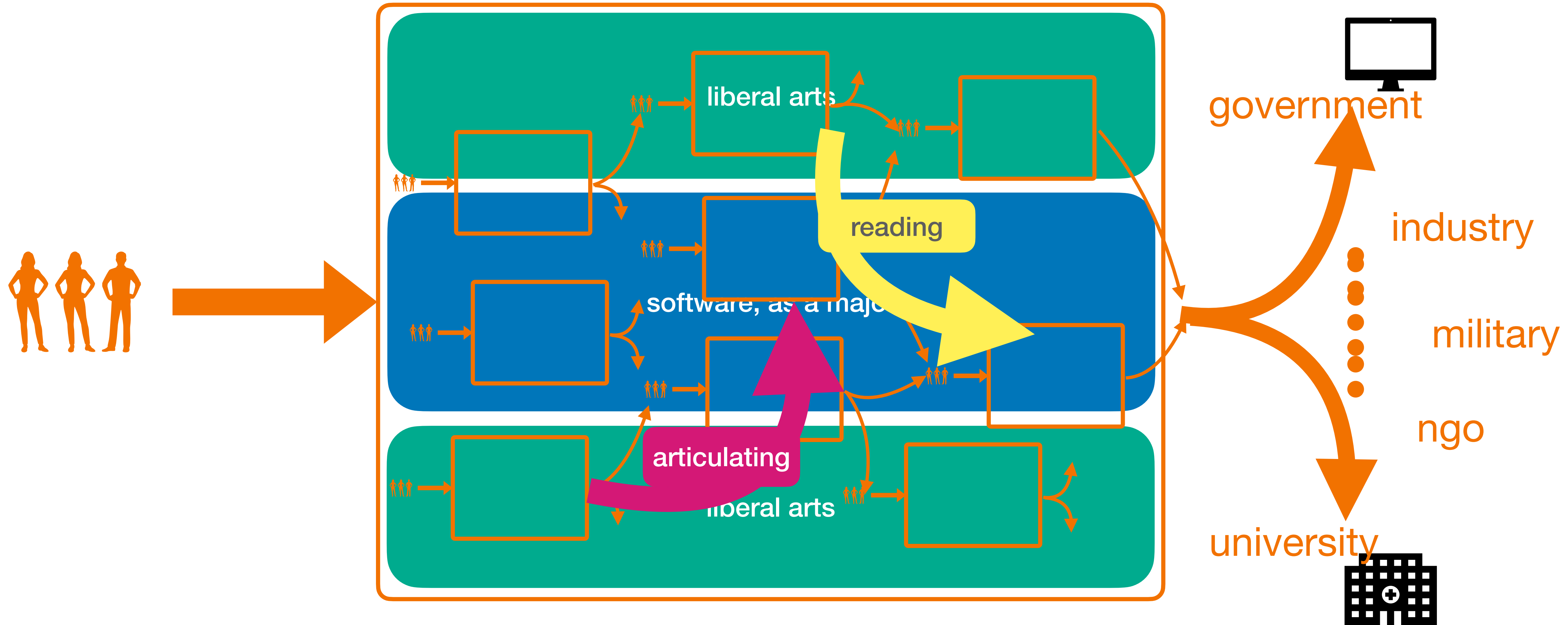
Is the Curriculum a Set?

Or Is It a Directed Graph?

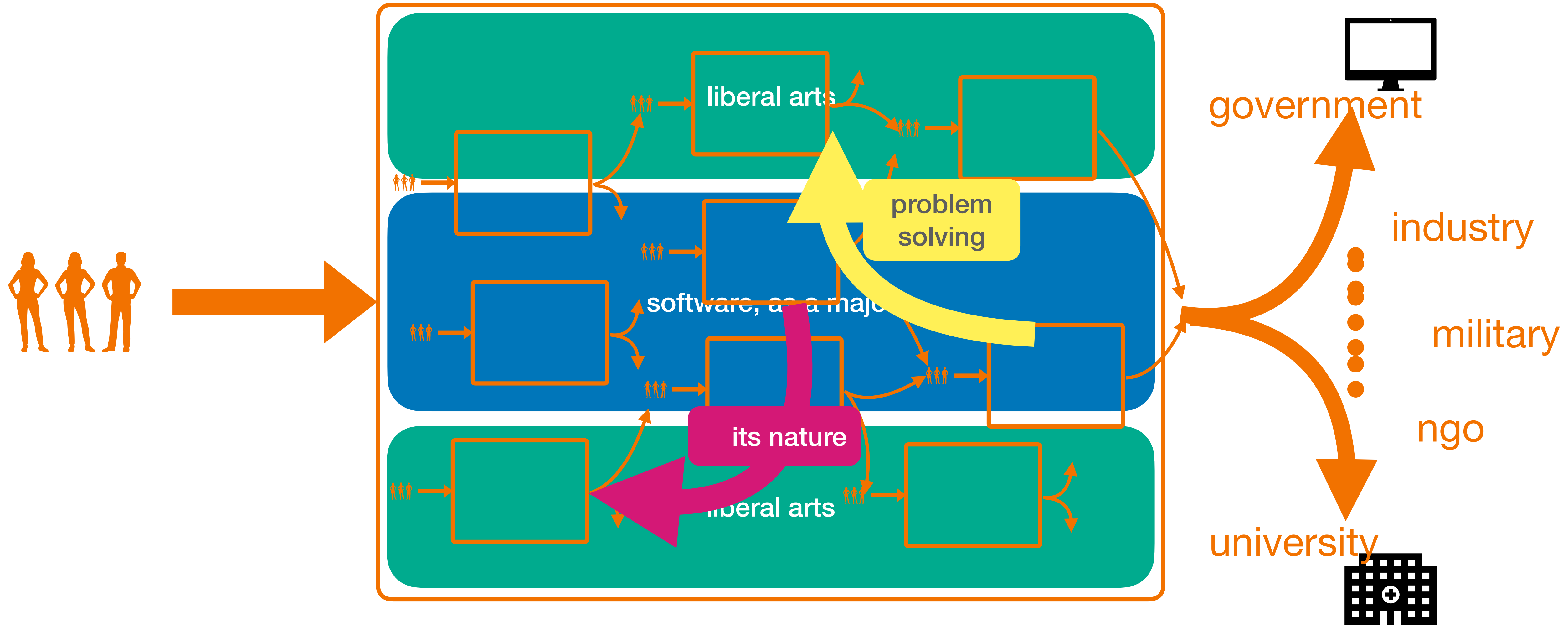
The College Curriculum, a Directed Graph



The College Curriculum



The College Curriculum



Basic Principles for a Software Major

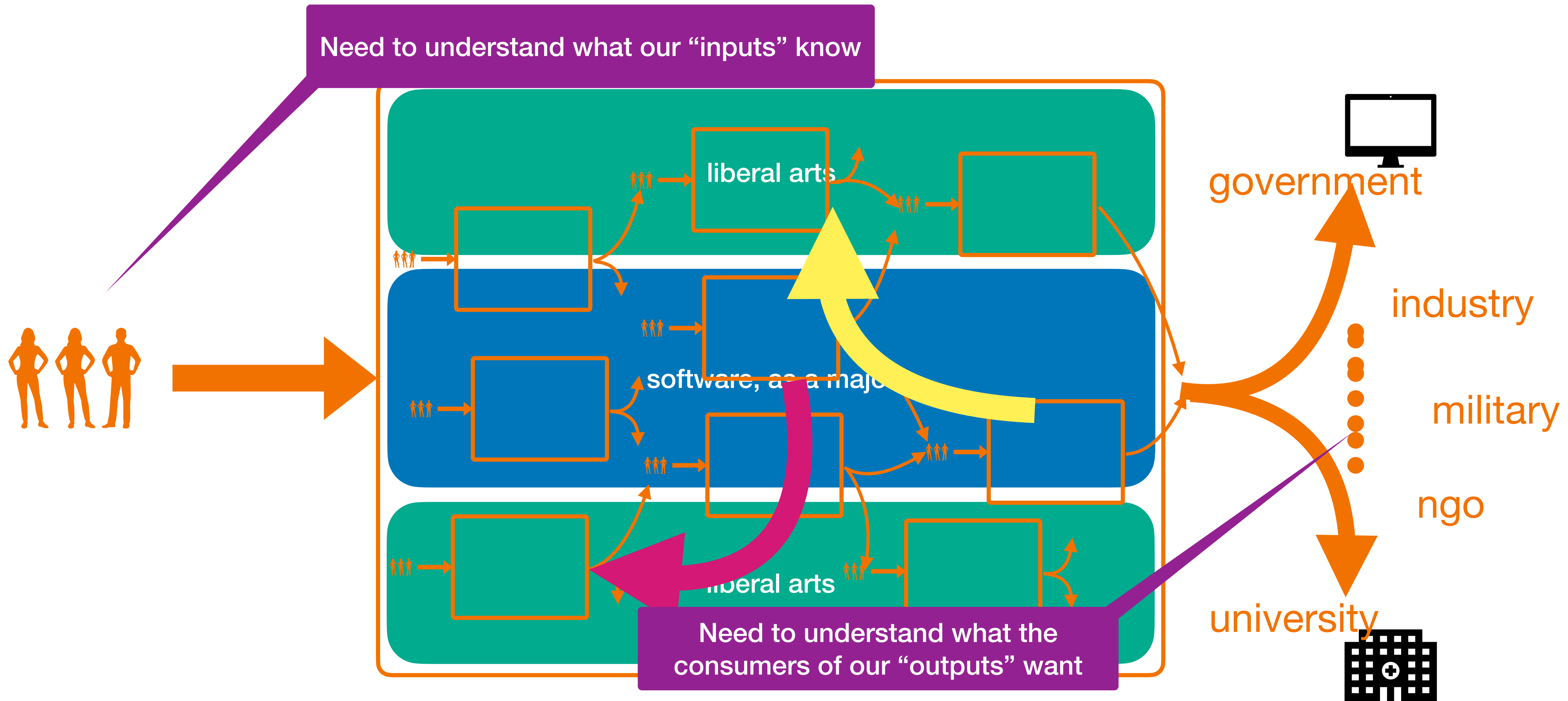
(1) Software development principles and ideas belong into the liberal arts curriculum.

(2) The liberal-arts approach to reading and articulating belong into the software curriculum.

What is Software?

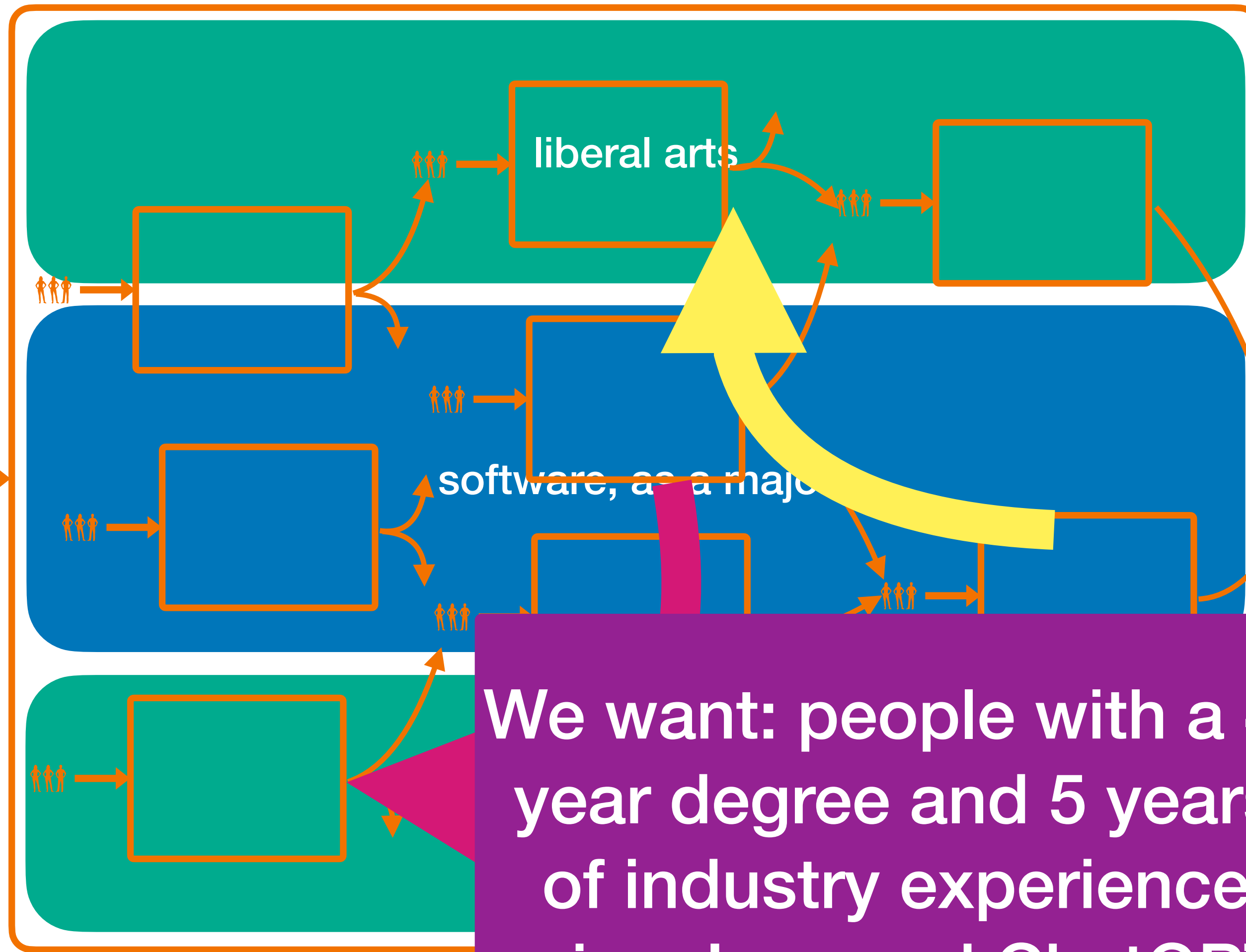
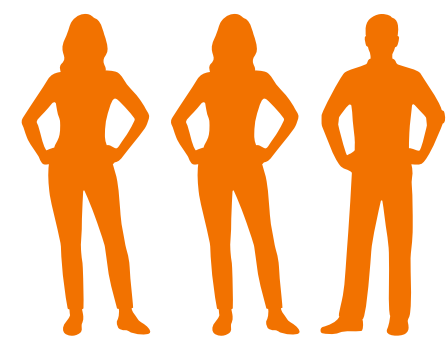
What is Software Design?

The College Curriculum

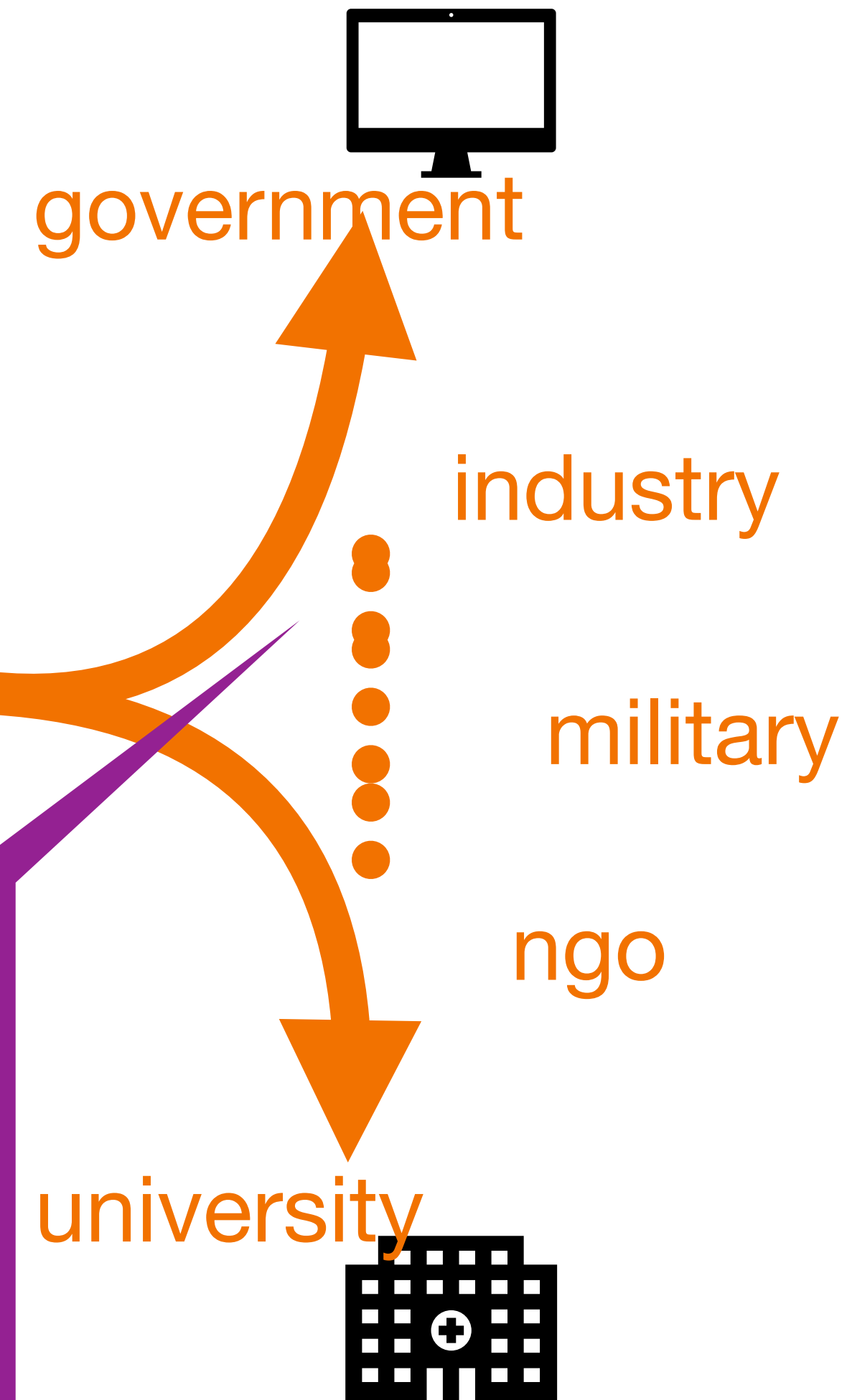


The College Curriculum

pre-algebra, very basic reading and writing

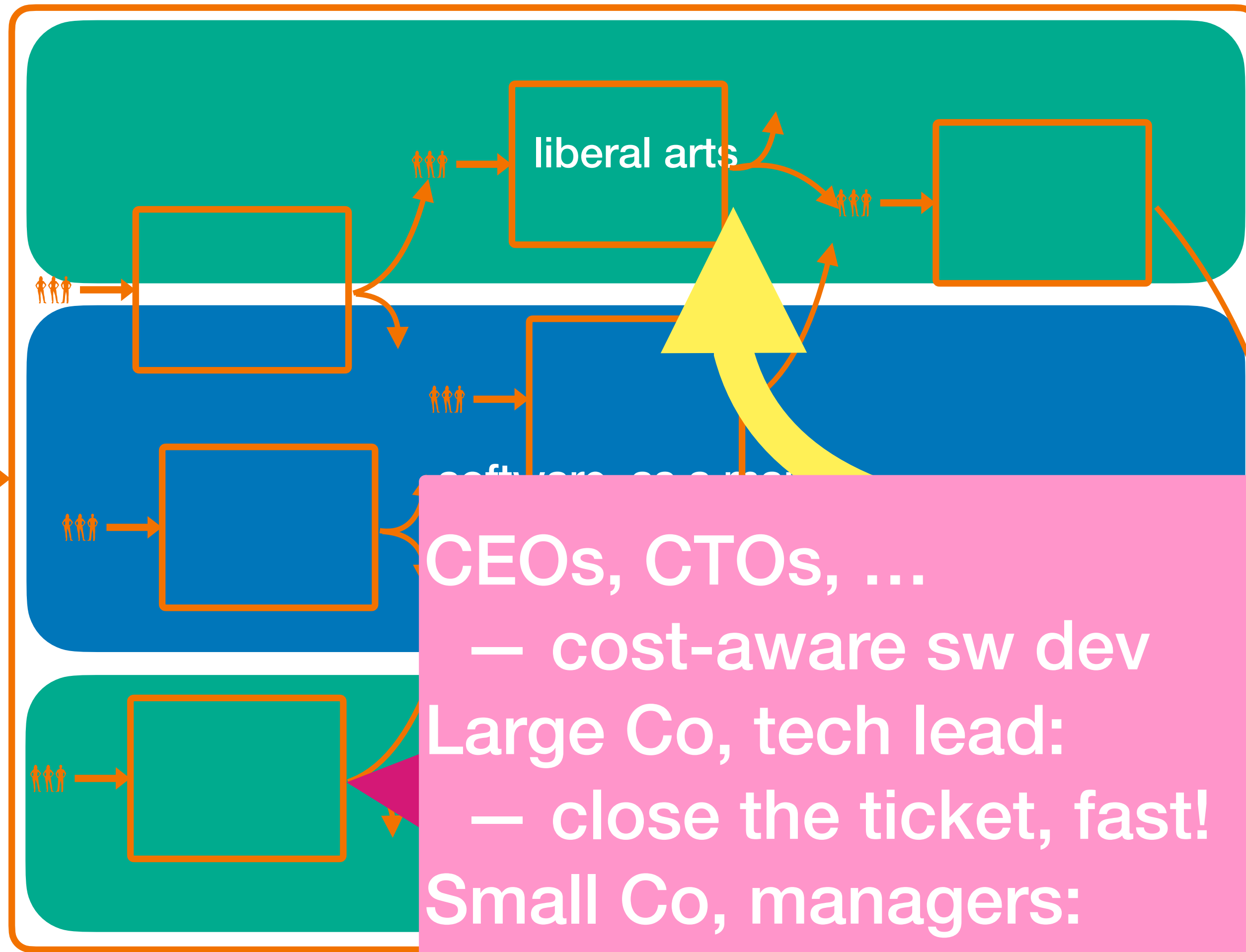
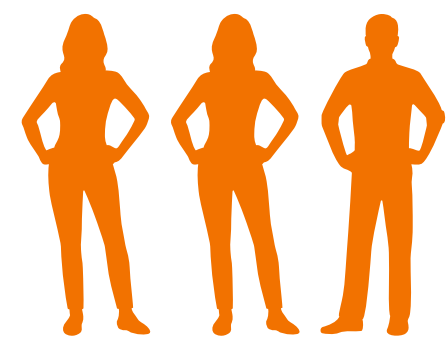


We want: people with a 4-year degree and 5 years of industry experience using Java and ChatGPT.



The College Curriculum

pre-algebra, English reading and writing



CEOs, CTOs, ...

— cost-aware sw dev

Large Co, tech lead:

— close the ticket, fast!

Small Co, managers:

— I can't spend too much on sw

government

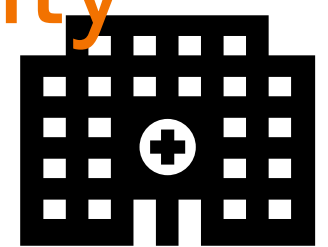


industry

military

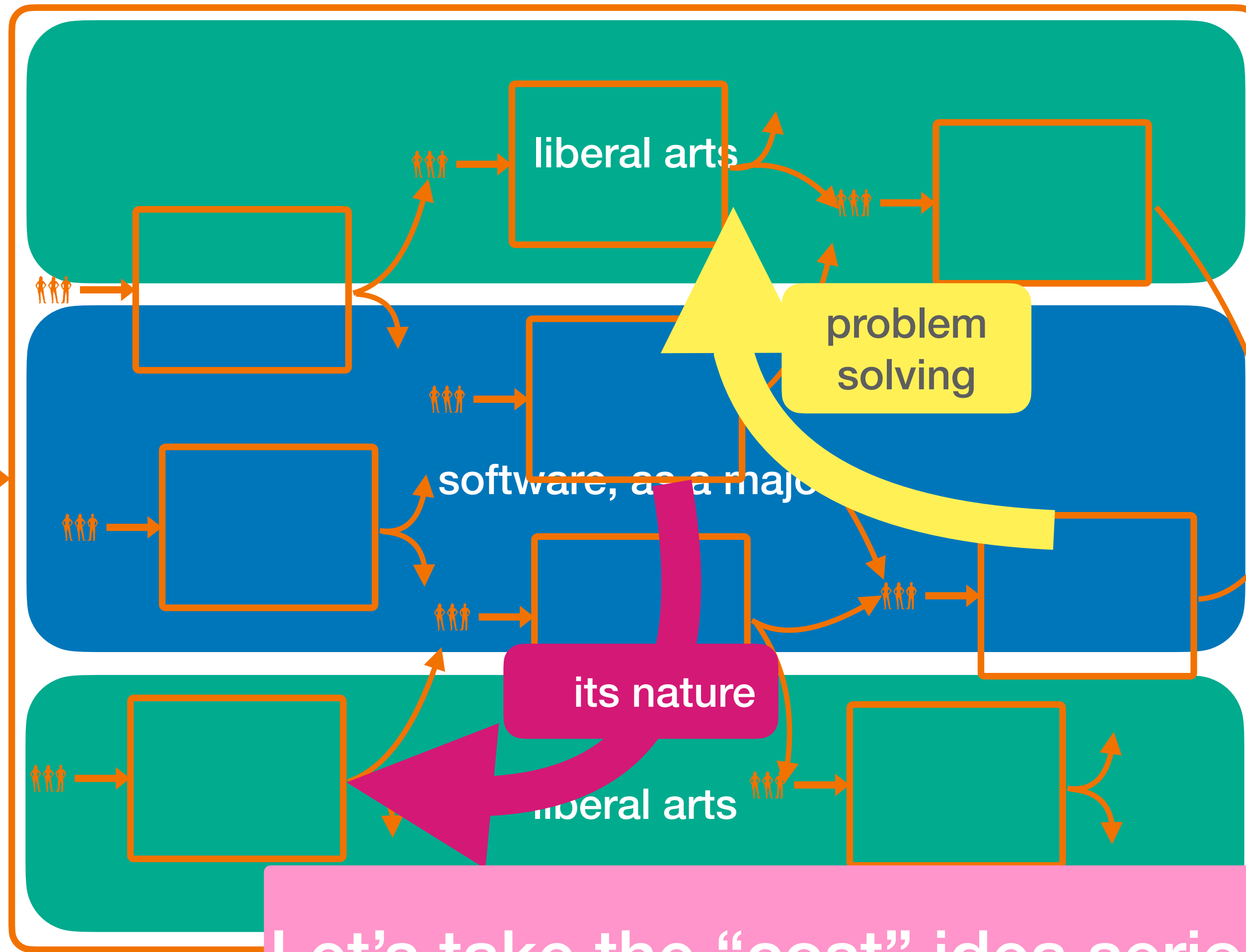
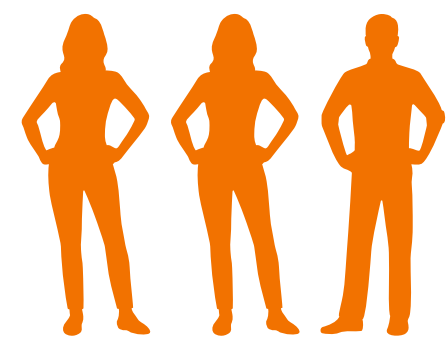
ngo

city

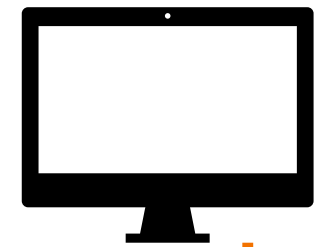


The College Curriculum

pre-algebra, English reading and writing



government

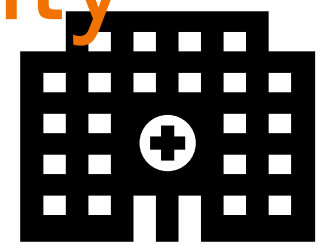


industry

military

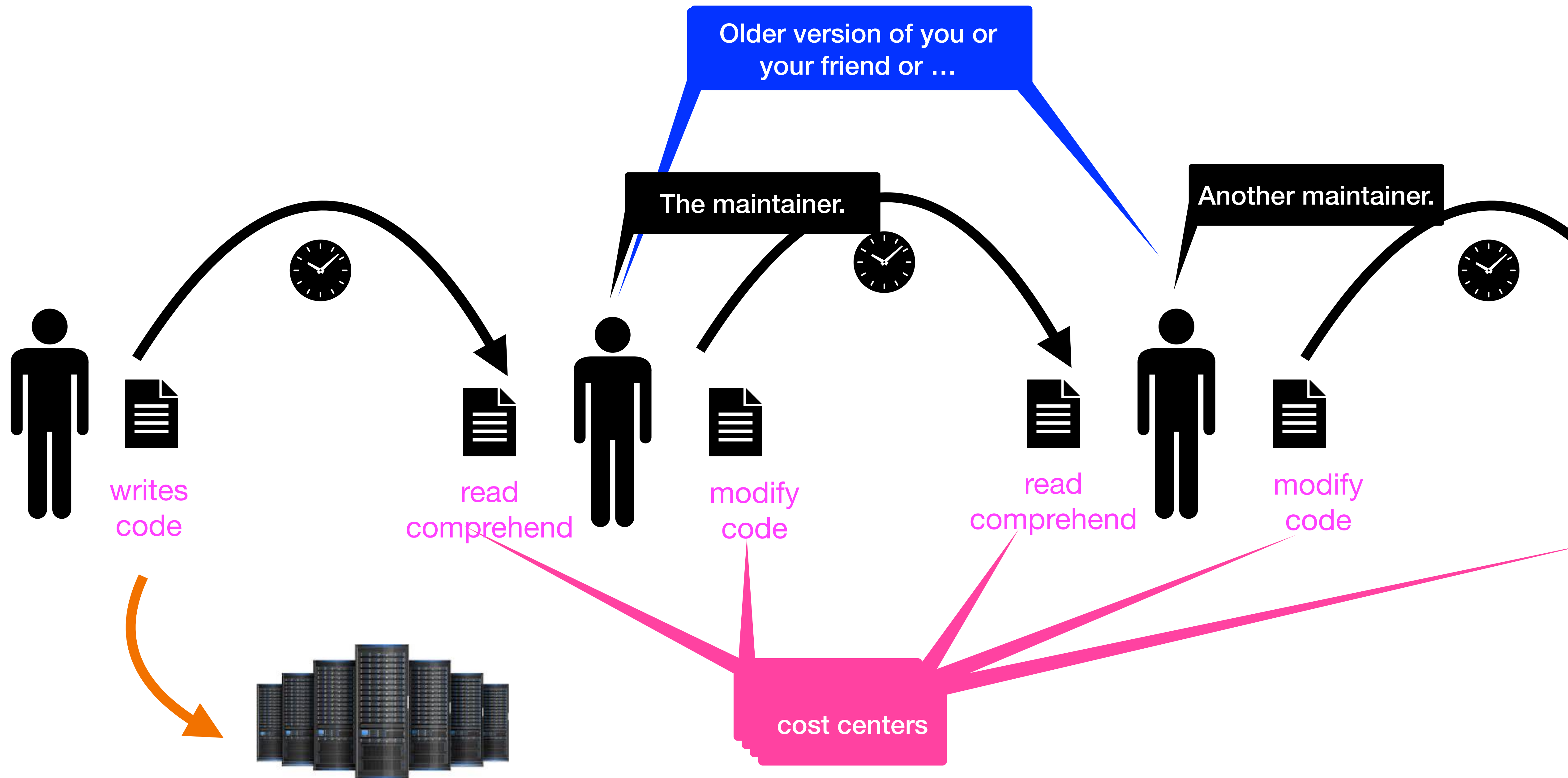
ngo

university



Let's take the "cost" idea seriously.

Software



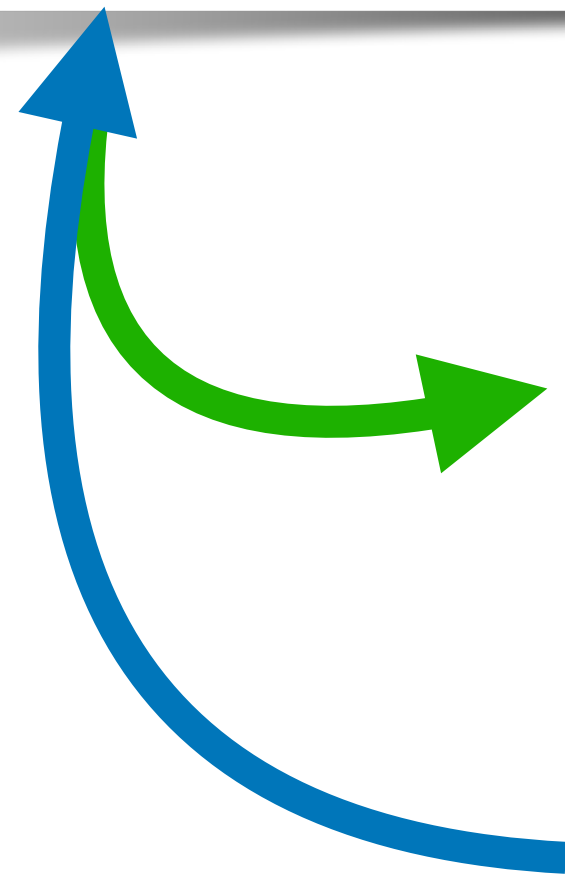
From the Perspective of Software,

(1) “Programming” is an undesirable skill.

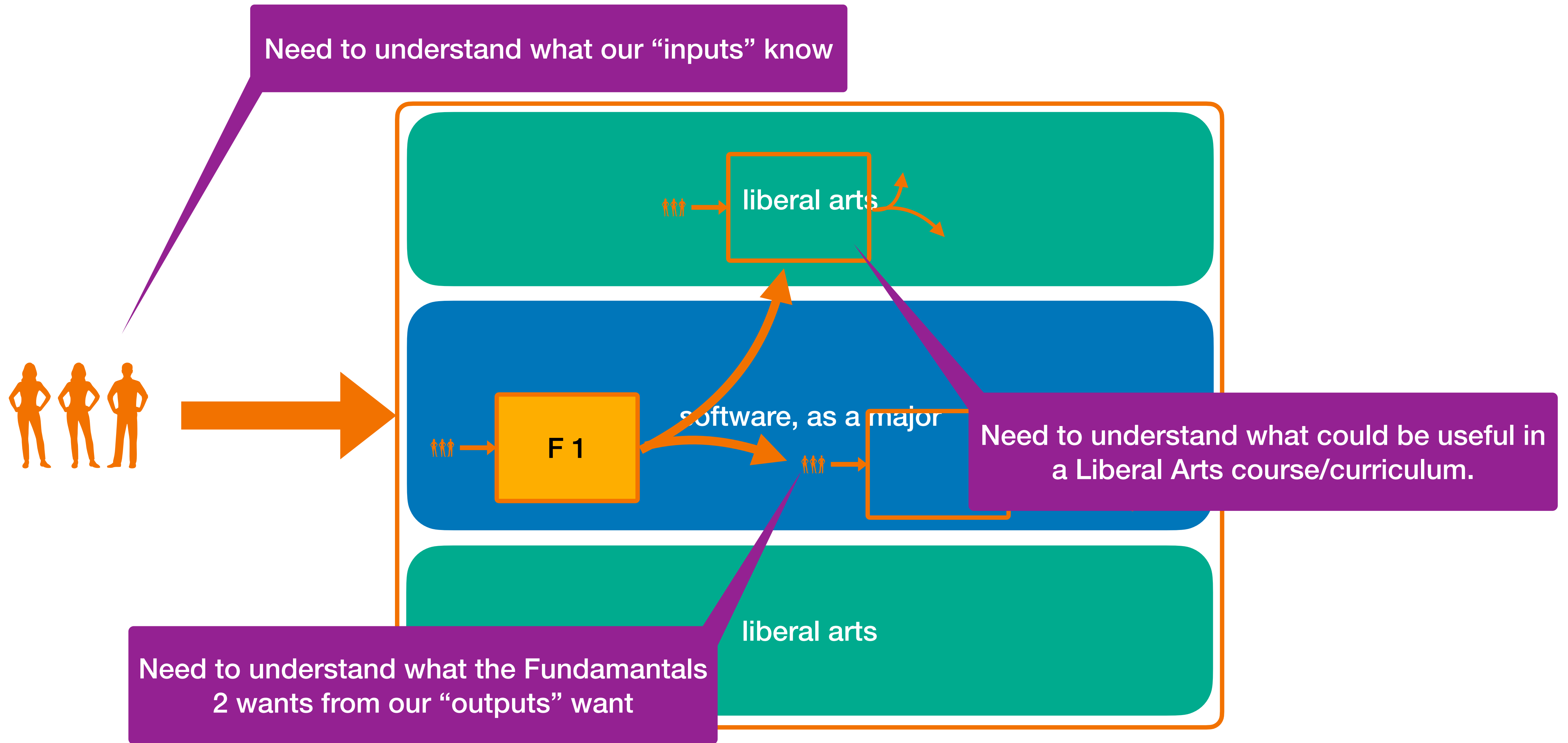
(2) “Systematic Program Design” is what we want.

(2a) .. problem solving for liberal arts students.

(2b) .. careful reading and articulating from the liberal arts for software students.

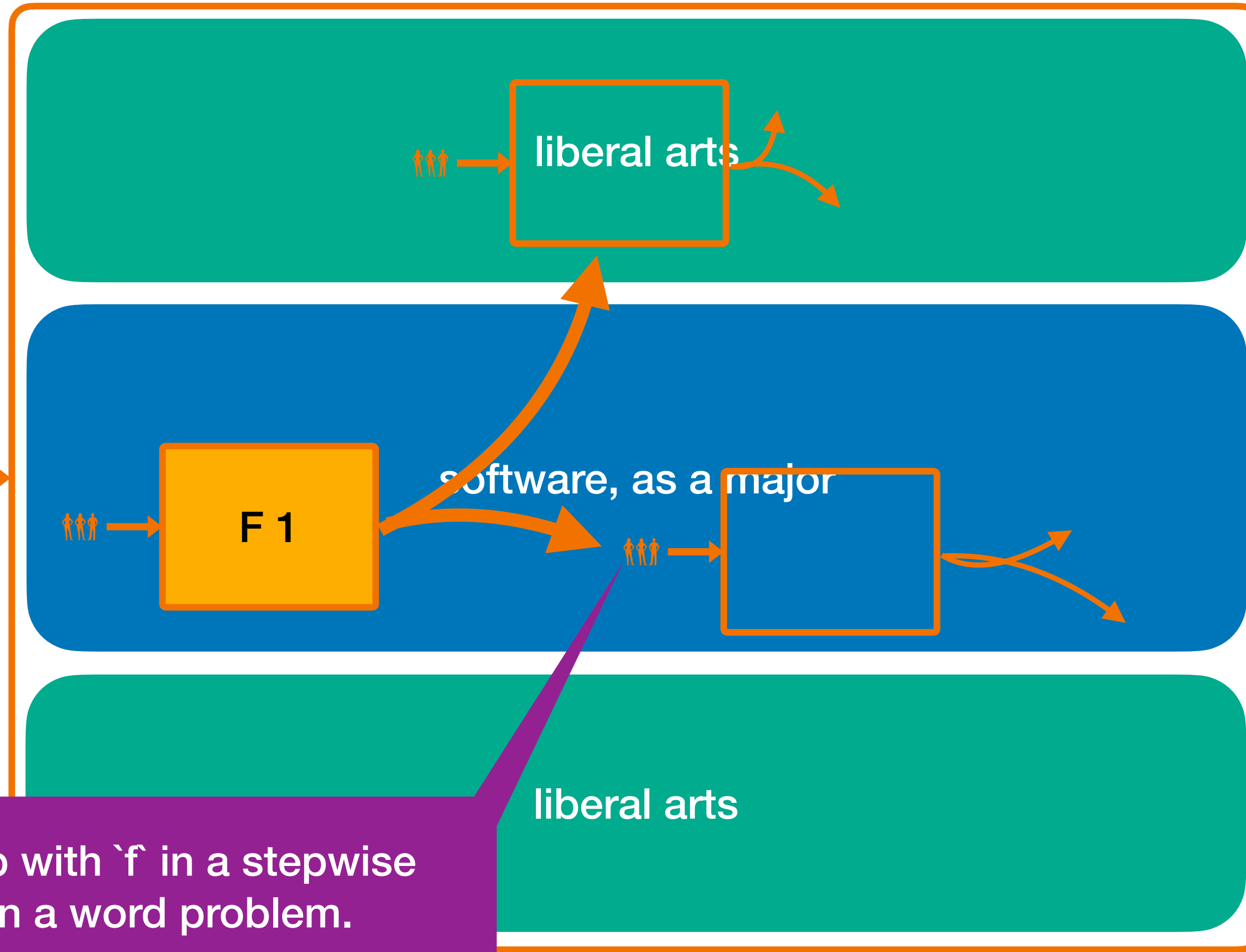
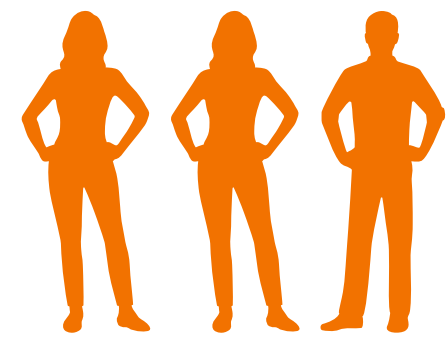


The Software Curriculum, Fundamentals 1



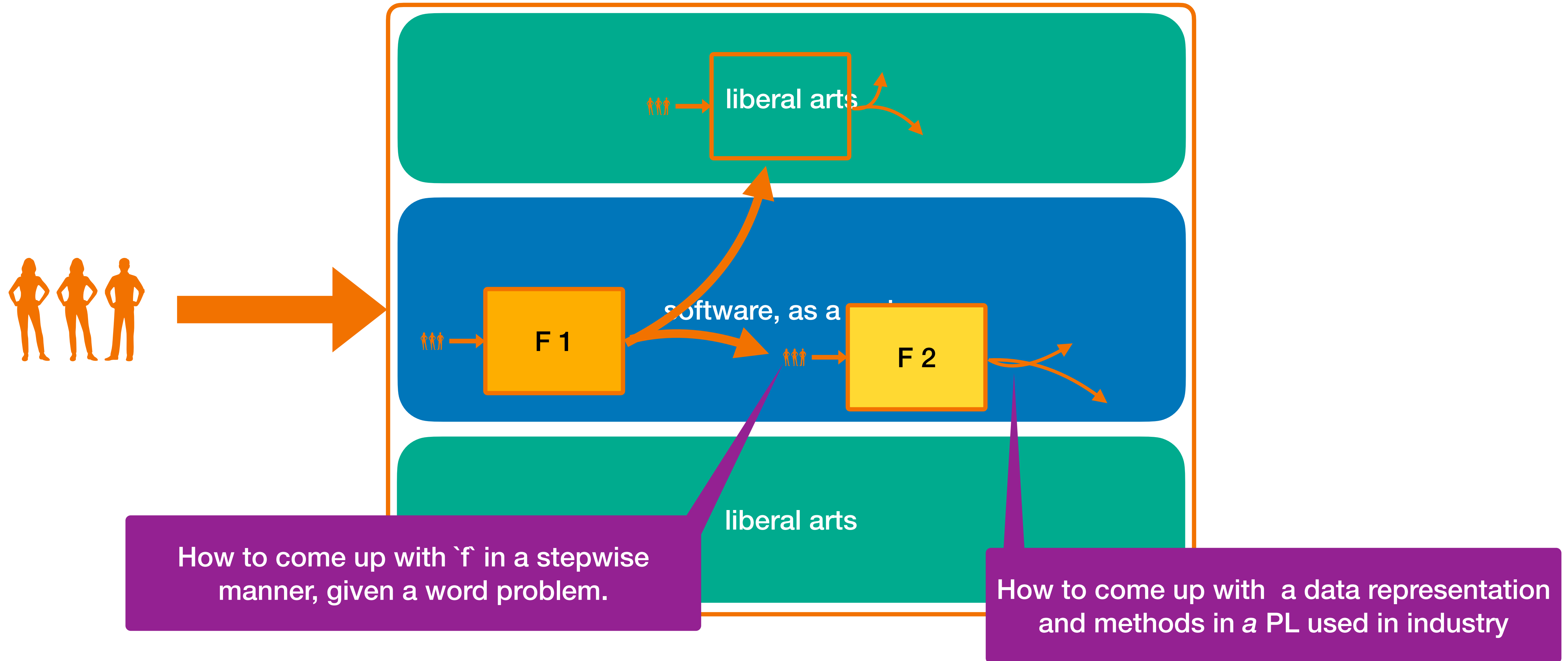
The Software Curriculum, Fundamentals 1

pre-algebra: $f(x) = \dots$, what is $f(55)$?
“Here is a problem. Create f ”.

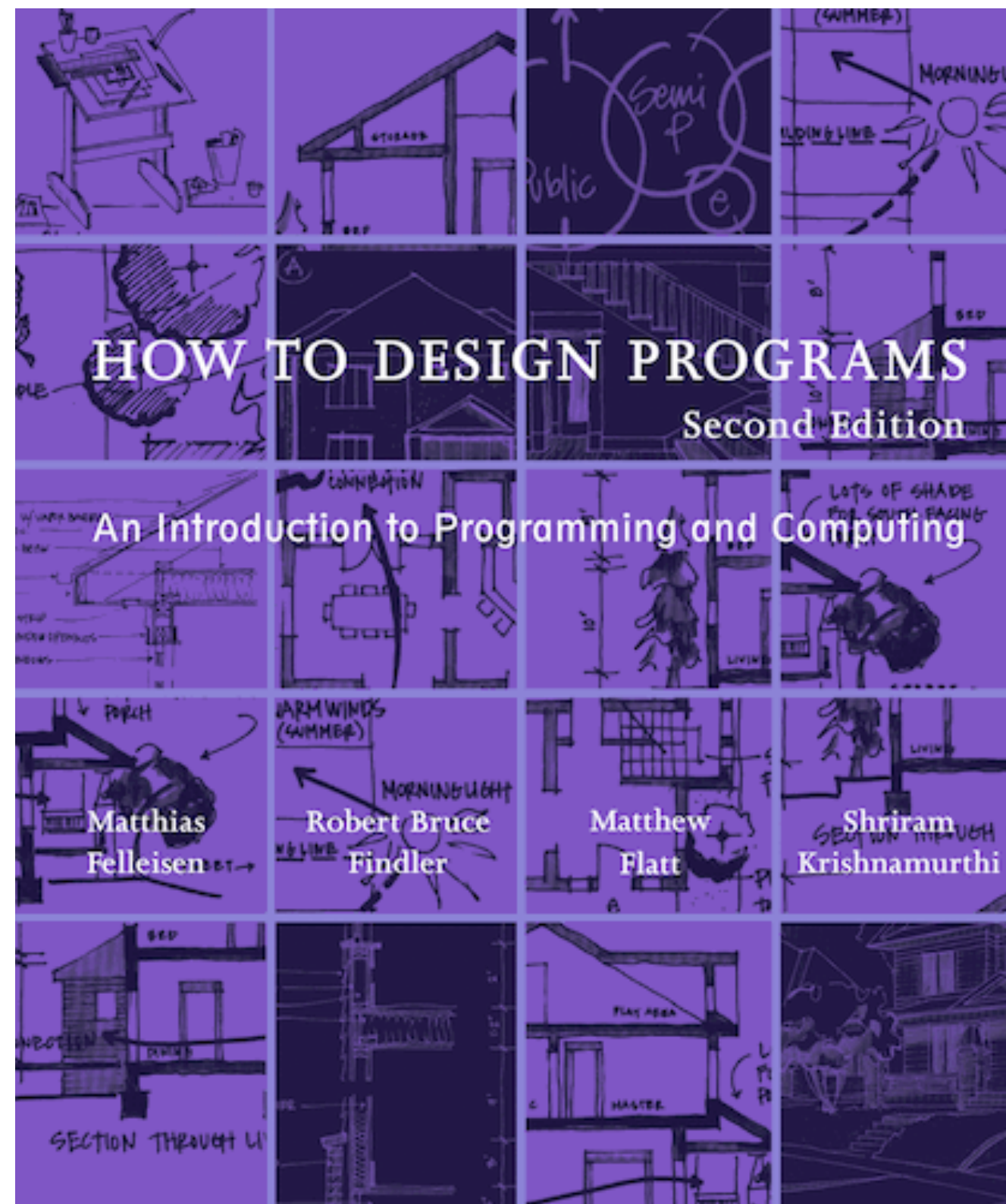


How to come up with f in a stepwise manner, given a word problem.

The Software Curriculum, Fundamentals 2



The Software Curriculum, the Fundamentals



How to Design Classes

Data, Structure and Organization

Matthias Felleisen
Matthew Flatt
Robert Bruce Findler
Kathryn E. Gray
Shriram Krishnamurthi
Viera K. Proulx

The Software Curriculum, the Fundamentals

F 1 has the goal to teach students how to solve word problems in a systematic manner (as code).

F 1 teaches a step-by-step approach, with each step producing a product that students and instructors inspect and critique.

F 1 teaches to split problems and to compose solutions, the key being the intermediate form of data.

F 1 teaches to an iterative refinement approach for complex and sophisticated word problems.

And btw, F 1 uses “pre algebra” as a PL, aka BSL ... not Racket.

The Software Curriculum, the Fundamentals

F 2 has the goal to teach students how to solve word problems in a systematic manner *with somewhat "industrial" OO, types.*

F 2 teaches a step-by-step approach, with each step producing a product that students and instructors can inspect and critique.

And btw, F 2 uses just the fragment of Java needed to bring across OO. It does not teach Java.

F 2 teaches to split problems and to compose solutions, the key to solving the intermediate form of data.

F 2 teaches to an iterative refinement approach for complex and sophisticated word problems.

The Software Curriculum, the Fundamentals

Step 1: Describe the information. Choose a data representation.

Learn to use classes.

Step 2: Articulate a focused purpose statement

It's the only line of comment you may use.

Step 3: Work through examples.

How else can you build intuition? Prepare tests?

Step 4: Take inventory. What data is available to compute the answer?

Did you write your essays without making an outline?

Step 5: Code.

Tests don't verify, but they do find bugs.
Proof reading anyone?

Step 6: Test. — Look back and evaluate

The Software Curriculum, the Fundamentals

How does one deal with complex problems?

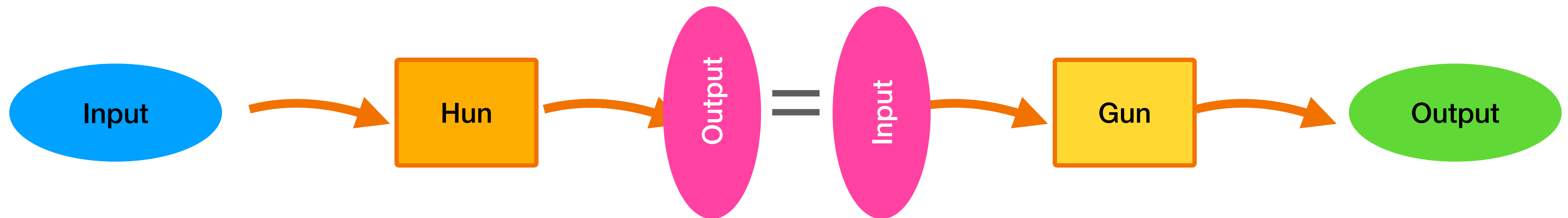
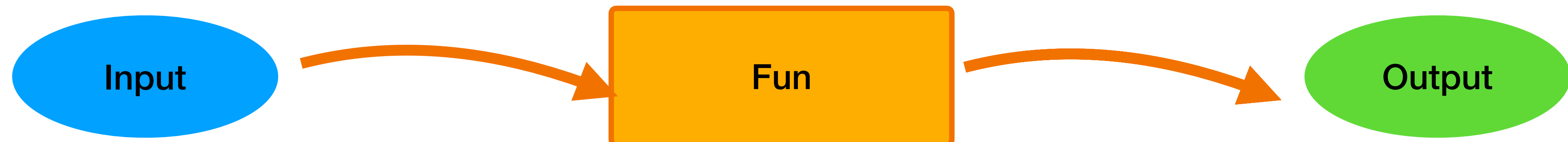
(1) *Composition*: create auxiliary functions, compose classes (*don't* inherit)

(2) *Iterative refinement*: add functionality, add data variants on piecemeal basis

The Software Curriculum, the Fundamentals

(1) *Composition*: create auxiliary functions, compose classes (*don't* inherit)

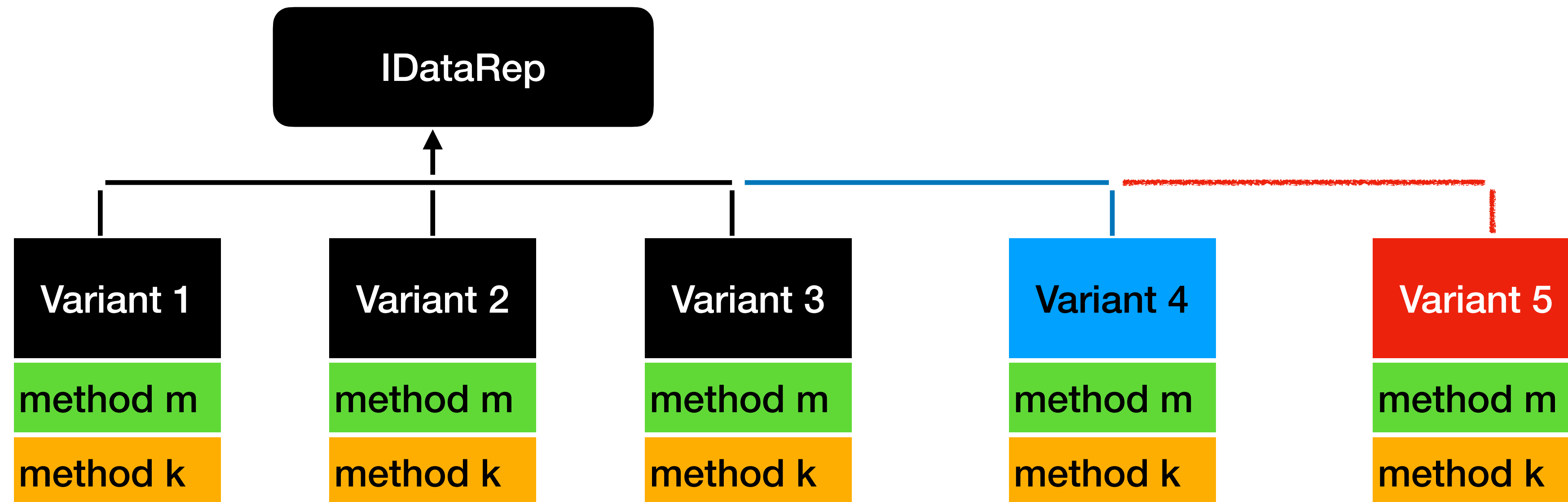
(1) Divide the problem. Compose the solutions. $f(g(x))$



The Software Curriculum, the Fundamentals

(2) *Iterative refinement*: add functionality, add data variants on piecemeal basis

(2) Make a hierarchical plan. Pick the most important goal. Solve. Repeat.



What is Curriculum?

What is Curriculum Design?

Curriculum Design, the Basic Steps

Step 1: Describe the information

Students know algebra. Consumers want Sw Dev professionals.

Step 2: ... purpose statement.

Students want Colleges to teach for (their professional) life.

Step 3: Work through examples.

Sw dev and the fundamentals they need (cost).

Step 4: Take inventory.

1968, 1980. We have learned a lot about sw dev in four decades. I know the literature. (Not CS Ed)

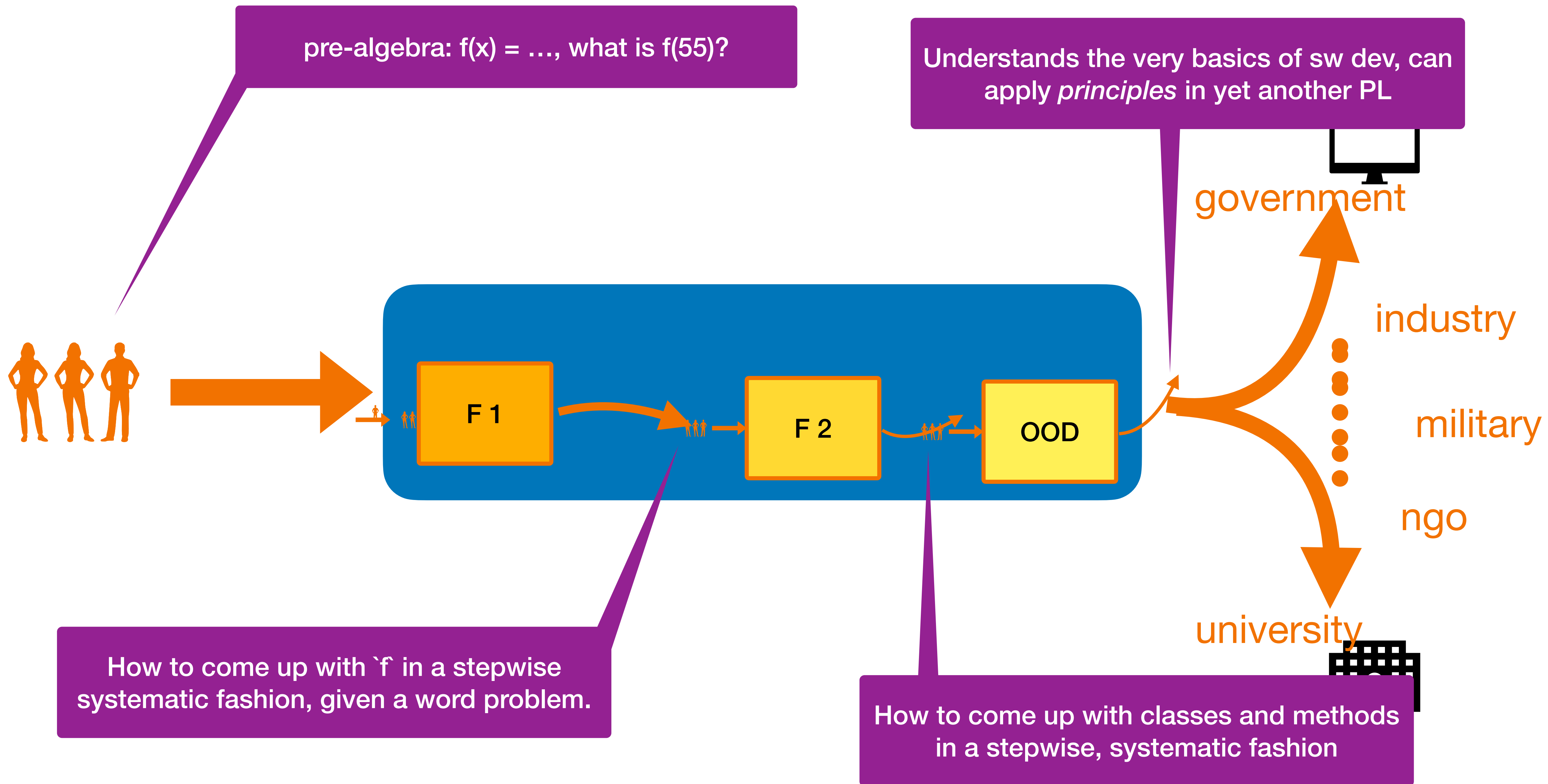
Step 5: Code.

Designing and teaching seven UG courses or working with others to implement them.

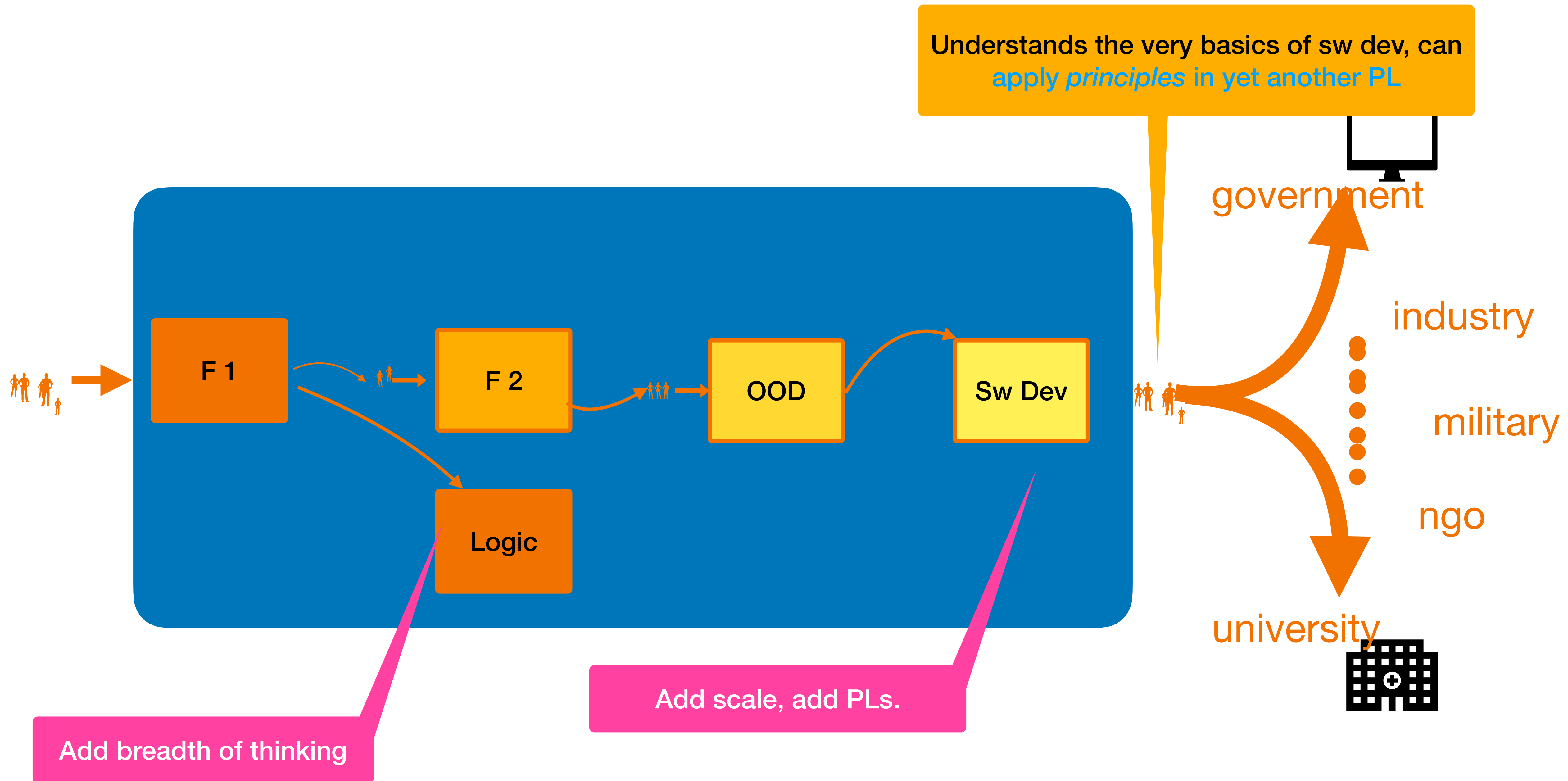
Step 6: Test. — Look back & evaluate.

Agnes, 2008, Harvard, recruiters, Google, ...

Curriculum Design, Course Composition



Curriculum Design, Iterative Refinement



What is *Liberal* Arts?

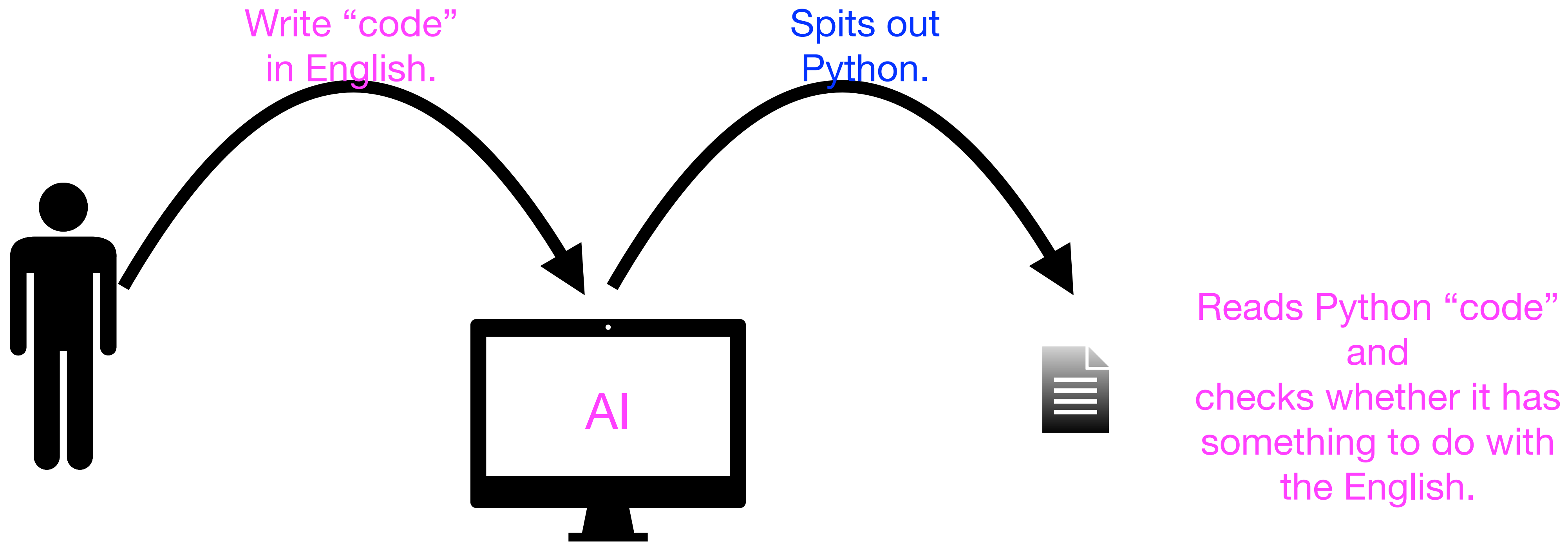
What is its Role in Software?

I wish you could ...

AI

- .. read
- .. write

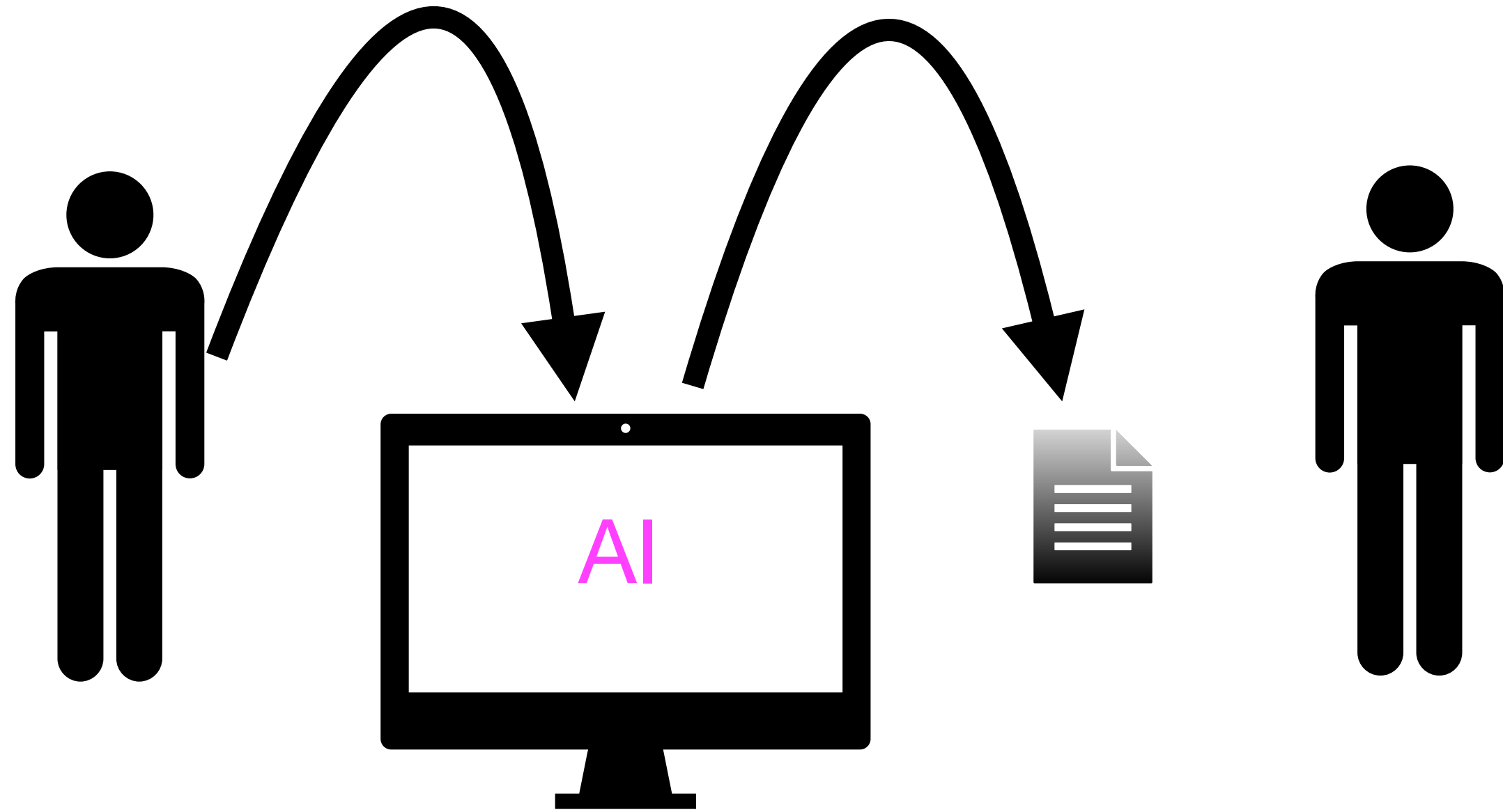
I wish you could read & write.



What if the co-worker is an AI?

I wish you could write.

What does it take to get an AI to create the code that we want?

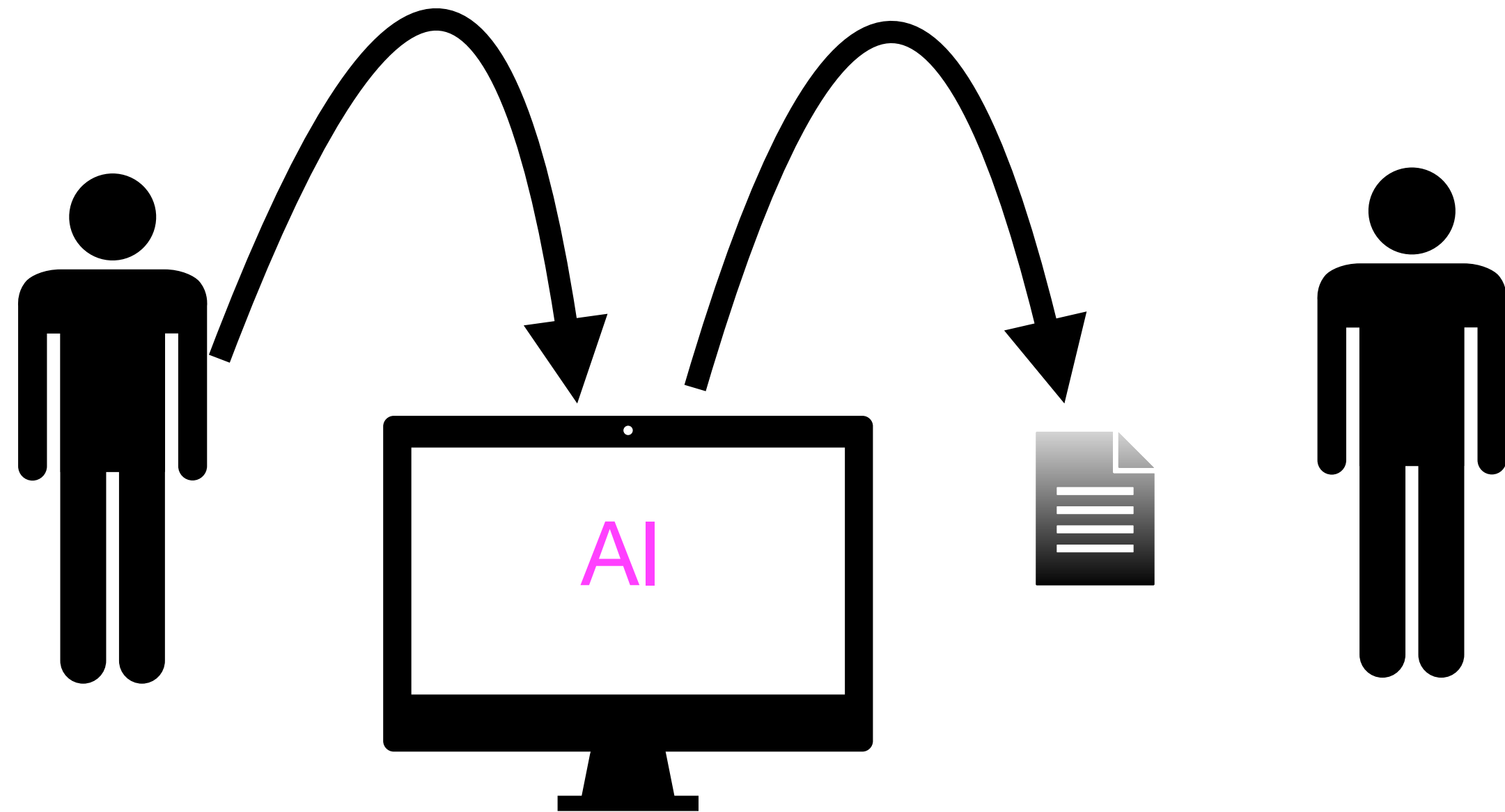


What if the co-worker is an AI?

- precise, focused *purpose statements*
- with a clear distinction between
 - atomic units of code
 - composite units of code
- a properly organized plan

I wish you could read.

What does it take to get an AI
to create *good* code?



What if the co-
worker is an AI?

Why should we doubt AI-generated code?

- bugs and git-reverts
- safety holes
- security holes

I wish you could read.

Study finds AI assistants help developers produce code that's more likely to be buggy

At the same time, tools like Github Copilot and Facebook InCoder make developers believe their code is sound

https://www.theregister.com/2022/12/21/ai_assistants_bad_code/

"We find disconcerting trends for maintainability. Code churn – the percentage of lines that are reverted or updated less than two weeks after being authored -- is projected to double in 2024 compared to its 2021, pre-AI baseline. We further find that the percentage of 'added code' and 'copy/pasted code' is increasing in proportion to 'updated,' 'deleted,' and 'moved' code. In this regard, AI-generated code resembles an itinerant contributor, prone to violate the DRY-ness [don't repeat yourself] of the repos visited."

New GitHub Copilot Research Finds 'Downward Pressure on Code Quality'
Visual Studio Magazine, 25 Jan 2024

I wish you could read.



by **Matt Asay**
Contributing Writer

in 

“Google’s Chrome team, writes, “AI tools help experienced developers more than beginners.”

Why AI coding assistants are best for experienced developers

6. Discussion

[Asleep at the Keyboard?](#)

[CACM 21 Jan 2025](#)

Copilot’s response to our scenarios is mixed from a security standpoint, given the large number of generated vulnerabilities (across all axes and languages, 39.33 % of the top and 40.73 % of the total options were vulnerable). The security of the top options are particularly important—novice users may be more likely to accept the ‘best’ suggestion.

How do I learn to read and write?

Read actual books, pay attention to details and nuances. Learn to write about books with the same attention to detail and nuance.



Take Aways.

How to Design a Curriculum

Software

- A curriculum is a program.
 - It consumes students and produces young professionals.
- A *good* curriculum is a *complex* program.
 - Compose courses like functions.
 - Each course consumes students and adds value.
 - The next course should rely on this value.
- The early stages of a software curriculum ought to teach widely valuable concepts.
- An *ideal* software curriculum gets support from the *Liberal Arts*, teaching
 - reading and writing
 - in a variety of contexts (of interest to the student).

The End

Credits:

Larry Finkelstein

Ben Lerner

Sam Tobin-Hochstadt

Jesse Tov

Daniel Patterson

Viera Proulx

Richard Rasala

David Van Horn

The Curriculum's Cross Connections

**Why Should a Software Developer
Know Anything about Kant, Parfit,
Faulkner, Mann, Hawking, Friedman,
Kahnemann, Bach, or Escher?**

**Why Should Someone in
Liberal Arts Know About
Software Development?**

The Curriculum's Cross Connections

**Why Should a Surgeon, Banker,
Journalist, Lawyer, Author Know
Anything Software Development?**

The College Curriculum

